OWL

Mikel Egaña Aranguren

mikel-egana-aranguren.github.io

mikel.egana@ehu.eus

Universidad del País Vasco
Euskal Herriko Unibertsitatea

BILBOKO INGENIARITZA ESKOLA
ESCUELA DE INGENIERÍA DE BILBAO

OWL

https://github.com/mikel-egana-aranguren/ABD

CC BY

OWL

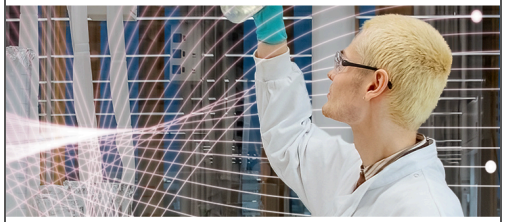
the LEGO Group

Lead Ontology Engineer - Semantic Data Modelling

- Proven expertise in Semantic Modelling and Ontology Design using open standards.
- Proficiency in RDF, OWL, and SHACL for developing and managing ontologies.
- Experience in data governance with knowledge graphs and semantic models.
- Strong programming skills in languages such as Python and TypeScript.
- Excellent communication and collaboration skills to work effectively within a multidisciplinary team.
- Understanding Domain Driven Design, loosely coupled components, and event driven architecture.
- Travel 15-20 days a year (dependent on need).
- Communicate effortlessly in English both written and verbally.

OWL

AstraZeneca Unirse a la Red de Talento



← Todos los Trabajos

Ontologist

Macclesfield, , United Kingdom

Aplicar Ahora | Añadir al Carro de Trabajo

Descripción del puesto vacante

Introduction to role

Join our Global Operations Data Office as an Ontologist and be part of a team that is transforming the pharmaceutical industry through data innovation.

Our mission is to ensure AI-ready data that supports our Operations 2030 ambition, enabling life-changing medicines. We partner with various business areas, including Pharmaceutical Technology and Development, Manufacturing, and Global Supply Chain, operating from hub sites in the UK, Sweden, US, China, and Global Technology Centres in India and Mexico. Here, your work will directly impact patients by empowering the business to perform at its peak through cutting-edge science and digital technology platforms.

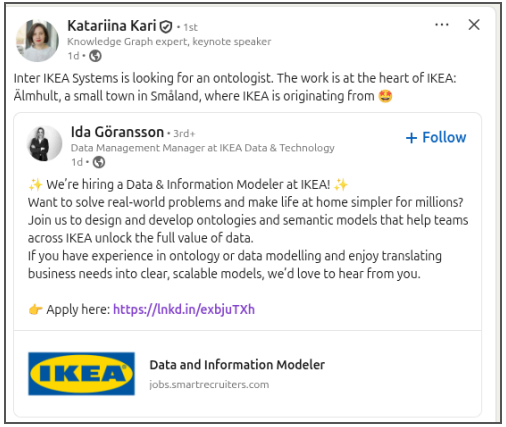
Accountabilities

As an Ontologist, you will play a crucial role in developing semantic data solutions across manufacturing, supply chain, and development functions. You will engineer robust ontology driven knowledge models, integrate structured and unstructured data, and apply semantic standards to enhance data interoperability. Collaborating closely with product teams and data engineers, you will bring semantic capabilities into real-world business applications. This is your chance to work at the forefront of semantic data technologies in the pharmaceutical industry—enabling FAIR data, accelerating AI use cases, and driving smarter digital operations.

Essential Skills/Experience

- Proficiency with RDF, SKOS, OWL, SHACL, SPARQL, and other semantic web technologies.

OWL


Katariina Kari • 1st
Knowledge Graph expert, keynote speaker
1d •
Inter IKEA Systems is looking for an ontologist. The work is at the heart of IKEA: Älmhult, a small town in Småland, where IKEA is originating from 🤩
Ida Göransson • 3rd+
Data Management Manager at IKEA Data & Technology
1d •
+ Follow
✨ We're hiring a Data & Information Modeler at IKEA! ✨
Want to solve real-world problems and make life at home simpler for millions?
Join us to design and develop ontologies and semantic models that help teams across IKEA unlock the full value of data.
If you have experience in ontology or data modelling and enjoy translating business needs into clear, scalable models, we'd love to hear from you.
👉 Apply here: https://lnkd.in/exbjuTXh
IKEA
Data and Information Modeler
jobs.smartrecruiters.com

OWL

OWL: Web Ontology Language

Estándar oficial del W3C para crear ontologías en la web con un semántica precisa y formal (=HTML, !=SQL)

Ontología: conjunto de descripciones precisas sobre un **dominio de interés**

Previene malentendidos en la comunicación humana

Asegura que el software se comporte de forma uniforme y predecible

OWL

El mismo lenguaje RDF para definir datos y su vocabulario* (NoSQL!RDF!)

Funcion 1: vocabulario para integración de datos

Funcion 2: lenguaje de modelado para describir el dominio de interés y aplicar razonamiento automático

OWL

No es un lenguaje de programación: es declarativo, describe un estado del dominio

No es un lenguaje de esquema: no prescribe la estructura de un documento (ej. XML/JSON schema) - Para eso esta SHACL

No es un framework de bases de datos: sigue la **Open World Assumption** (un hecho ausente puede ser verdadero, no se asume como falso)

Closed World vs Open World

Closed World Assumption (CWA): lo que no está en la BD es falso (SQL)

Open World Assumption (OWA): lo que no se dice puede ser verdadero o falso (OWL)

Ejemplo: si no hay info sobre la edad de María, SQL asume NULL/sin edad; OWL asume que simplemente no se sabe

Open World Assumption (OWA)

La ausencia de información no implica falsedad

Si no sabemos que Mikel vive en Bilbao, no asumimos que vive en otro sitio

Diferente de bases de datos relacionales (Closed World)

Consecuencia: las restricciones universales no son restrictivas por defecto

Para "cerrar" el mundo: usar propiedades funcionales, closure axioms, SHACL

No Unique Name Assumption (NUNA)

Dos URIs diferentes pueden referirse a la misma entidad

ex:Bilbao y ex:Bilbo pueden ser owl:sameAs

Útil para integración de datos de diferentes fuentes

Permite reconciliación automática mediante razonamiento

owl:differentFrom para forzar distinción

OWL

Entidades: los elementos que se refieren a objetos del dominio: *Juan, Persona, tieneEdad, ...*

Axiomas: las afirmaciones básicas de una ontología (se consideran verdaderas): *Juan es una Persona, Los adultos tienen edad mayor a 18, ...*

Expresiones: combinaciones de entidades y axiomas para formar descripciones complejas: *un funcionario es un adulto que tiene por lo menos un trabajo de duracion indefinida en una institucion que recibe fondos ...*

Entidades

Individuos (individuals): objetos concretos (ej: Juan, María)

Clases (classes): categorías que agrupan individuos (ej: Persona, Mujer)

Propiedades (properties): relaciones entre entidades

Propiedades

- **Object properties**: relacionan objeto con objeto (ej: tienePareja)
- **Datatype properties**: asignan valores de datos a objetos (ej: tieneEdad)
- **Annotation properties**: metadatos sobre la ontología (ej: rdfs:comment)

Razonamiento

Los **razonadores** (reasoners) pueden calcular automáticamente **consecuencias** de los axiomas

Un conjunto de axiomas A **implica** (entails) un axioma a si a es verdadero siempre que A es verdadero

Un conjunto de axiomas puede ser **consistente** (hay un estado posible donde todos son verdad) o **inconsistente**

Sintaxis OWL 2

Functional-Style Syntax: para especificación e implementación

RDF/XML: obligatoria para todas las herramientas OWL 2

Turtle: serialización RDF legible

Manchester Syntax: diseñada para no-lógicos

OWL/XML: basada en XML Schema

Clases, Propiedades e Individuos

Modelado básico con OWL 2

[protege]

Clases e instancias

Declarar que un individuo pertenece a una clase:

```
:Mary rdf:type :Person .
:Mary rdf:type :Woman .
```

[OWL RDF Turtle syntax]

Un individuo puede pertenecer a varias clases simultáneamente (RDF! Web Semantica! NoSQL!)

Jerarquía de clases (SubClassOf)

"Toda mujer es persona" → relación subclase

```
:Woman rdfs:subClassOf :Person .
:Mother rdfs:subClassOf :Woman .
```

Clases equivalentes

Dos clases contienen exactamente los mismos individuos:

```
:Person owl:equivalentClass :Human .
```

Equivale a declarar SubClassOf en ambas direcciones

Disjunción de clases (DisjointClasses)

Ningún individuo puede pertenecer a ambas clases a la vez:

```
:Woman owl:disjointWith :Man .
```

Si María es Mujer, el razonador infiere que María **no** es Hombre

Object Properties

Relacionan individuos entre sí:

```
:Jon rdf:type owl:NamedIndividual ;
    :pareja :Mary .

[ rdf:type owl:NegativePropertyAssertion ;
  owl:sourceIndividual :Jon ;
  owl:assertionProperty :pareja ;
  owl:targetIndividual :Mary
] .
```

Jerarquía de propiedades

```
:pareja rdfs:subPropertyOf :relacion .
```

Si B es pareja de A → B tiene relacion con A

(La relación inversa no es necesariamente cierta)

Domain y Range

Restricciones sobre qué tipos de entidades conecta una propiedad:

```
:tieneHijo rdfs:domain :persona .
:tieneHijo rdfs:range :menor .
```

Si "Pedro tieneHijo Maria" → Pedro es Persona, Maria es menor

⚠ Domain/Range **no son restricciones**, son **axiomas de inferencia**

⚠ Error común con Domain/Range

Ejemplo: edad domain Rersona

Si decimos "Felix edad 9" y "Felix rdf:type Gato"

→ El razonador infiere que Felix **también** es Persona

Domain/Range no validan: **infieren** nuevas clases

(Solucion?)

Igualdad y desigualdad de individuos

OWL **no asume** que nombres diferentes = individuos diferentes (No Unique Name Assumption)

```
:James owl:sameAs :Jim .
:John owl:differentFrom :Bill .
```

Datatype Properties

Relacionan individuos con valores de datos (literales):

```
:John :hasAge "51"^^xsd:integer .

[] a owl:NegativePropertyAssertion ;
  owl:sourceIndividual :Jack ;
  owl:assertionProperty :hasAge ;
  owl:targetValue "53"^^xsd:integer .

:hasAge rdfs:domain :Person .

:hasAge rdfs:range xsd:nonNegativeInteger .
```

datatypes de XML Schema: xsd:integer, xsd:string, xsd:boolean, …

Relaciones avanzadas de clases

Constructores lógicos para clases complejas

Intersección de clases (AND)

La clase contiene individuos que pertenecen a **ambas** clases:

```
EquivalentClasses(:Mother ObjectIntersectionOf(:Parent :Woman)) ] .
```

[OWL functional syntax]

→ Toda Mother es Woman y toda Mother es Parent

Unión de clases (OR)

La clase contiene individuos que pertenecen a **al menos una** de las clases:

```
Class: Parent
  EquivalentTo:
    Father or Mother
```

[Manchester OWL syntax]

Complemento de clases (NOT)

Los individuos que **no** pertenecen a la clase:

```
# ChildlessPerson = Person AND NOT Parent

:ChildlessPerson owl:equivalentClass [
  owl:intersectionOf ( :Person [
    rdf:type owl:Class ;
    owl:complementOf :Parent
  ] )
] .
```

Los constructores se pueden anidar arbitrariamente

SubClassOf con expresiones complejas

Condiciones necesarias pero no suficientes:

```
# Todo Grandfather es Man AND Parent
# (pero no todo Man AND Parent es Grandfather)
SubClassOf(
  :Grandfather
  ObjectIntersectionOf( :Man :Parent )
)
```

EquivalentClasses = condición **necesaria y suficiente**

SubClassOf = solo condición **necesaria**

Restricción existencial (∃, SOME)

Al menos un valor relacionado pertenece a una clase:

```
# Parent = tiene al menos un hijo que es Person
EquivalentClasses(
  :Parent
  ObjectSomeValuesFrom( :hasChild :Person )
)
```

Indicadores en lenguaje natural: "algún", "al menos uno"

Restricción universal (∀, ONLY)

Todos los valores relacionados pertenecen a una clase:

```
# HappyPerson = todos sus hijos son HappyPerson
EquivalentClasses(
  :HappyPerson
  ObjectAllValuesFrom( :hasChild :HappyPerson )
)
```

Indicadores: "solo", "exclusivamente", "nada más que"

⚠ Trampa de la restricción universal

ObjectAllValuesFrom(:hasChild :HappyPerson)

¡Una persona **sin hijos** cumple esta restricción!

Para exigir que además tenga al menos un hijo feliz:

```
EquivalentClasses(
  :HappyPerson
  ObjectIntersectionOf(
    ObjectAllValuesFrom( :hasChild :HappyPerson )
    ObjectSomeValuesFrom( :hasChild :HappyPerson )
  )
)
```

HasValue y HasSelf

```
# Hijos de John = los que tienen padre John
EquivalentClasses(
  :JohnsChildren
  ObjectHasValue( :hasParent :John )
)

# Narcisista = se ama a sí mismo
EquivalentClasses(
  :NarcisticPerson
  ObjectHasSelf( :loves )
)
```

Restricciones de cardinalidad

Especificar el número de individuos relacionados:

```
# John tiene como máximo 4 hijos que son padres
ClassAssertion(
  ObjectMaxCardinality( 4 :hasChild :Parent )
  :John )

# John tiene al menos 2 hijos que son padres
ClassAssertion(
  ObjectMinCardinality( 2 :hasChild :Parent )
  :John )

# John tiene exactamente 3 hijos que son padres
ClassAssertion(
  ObjectExactCardinality( 3 :hasChild :Parent )
  :John )
```

Restricciones de cardinalidad: Turtle

```
# Exactamente 5 hijos
:John rdf:type [
  rdf:type owl:Restriction ;
  owl:onProperty :hasChild ;
  owl:cardinality "5"^^xsd:nonNegativeInteger
] .

# Máximo 4 hijos que son Parent
:John rdf:type [
  rdf:type owl:Restriction ;
  owl:onProperty :hasChild ;
  owl:maxQualifiedCardinality
    "4"^^xsd:nonNegativeInteger ;
  owl:onClass :Parent
] .
```

Enumeración de individuos (OneOf)

Definir una clase enumerando todos sus miembros:

```
# Clase cerrada: solo Bill, John y Mary
EquivalentClasses(
  :MyBirthdayGuests
  ObjectOneOf( :Bill :John :Mary )
)

# Turtle:
:MyBirthdayGuests owl:equivalentClass [
  owl:oneOf ( :Bill :John :Mary )
] .
```

Si Jeff es MyBirthdayGuests → Jeff = Bill, John o Mary

Uso avanzado de propiedades

Características y construcciones avanzadas

Propiedades inversas

```
# hasParent es la inversa de hasChild
InverseObjectProperties( :hasParent :hasChild )

# Turtle:
:hasParent owl:inverseOf :hasChild .

# Uso anónimo en expresiones:
EquivalentClasses(
  :Orphan
  ObjectAllValuesFrom(
    ObjectInverseOf( :hasChild )
    :Dead
  )
)
```

Propiedades simétricas y asimétricas

```
# Si A hasSpouse B → B hasSpouse A
SymmetricObjectProperty( :hasSpouse )

# Si A hasChild B → NUNCA B hasChild A
AsymmetricObjectProperty( :hasChild )

# Turtle:
:hasSpouse rdf:type owl:SymmetricProperty .
:hasChild rdf:type owl:AsymmetricProperty .
```

Asimétrica ≠ no-simétrica (asimétrica es más fuerte)

Propiedades reflexivas e irreflexivas

```
# Todo individuo es su propio pariente
ReflexiveObjectProperty( :hasRelative )

# Nadie puede ser su propio padre
IrreflexiveObjectProperty( :parentOf )

# Turtle:
:hasRelative rdf:type owl:ReflexiveProperty .
:parentOf rdf:type owl:IrreflexiveProperty .
```

Propiedades funcionales

```
# Cada persona tiene como máximo un marido
FunctionalObjectProperty( :hasHusband )

# Cada marido lo es de como máximo una persona
InverseFunctionalObjectProperty( :hasHusband )

# Turtle:
:hasHusband rdf:type owl:FunctionalProperty .
:hasHusband rdf:type owl:InverseFunctionalProperty .
```

Si Mary hasHusband James y Mary hasHusband Jim → James = Jim

Propiedades transitivas

```
# Si A hasAncestor B y B hasAncestor C
# → A hasAncestor C
TransitiveObjectProperty( :hasAncestor )

# Turtle:
:hasAncestor rdf:type owl:TransitiveProperty .
```

hasParent es subpropiedad de hasAncestor: los padres de padres de padres se incluyen automáticamente

Propiedades disjuntas

```
# Nadie es padre e hijo de la misma persona
DisjointObjectProperties( :hasParent :hasSpouse )
```

No puede haber dos individuos conectados simultáneamente por ambas propiedades

Cadenas de propiedades (Property Chains)

```
# hasParent o hasParent = hasGrandparent
SubObjectPropertyOf(
  ObjectPropertyChain( :hasParent :hasParent )
  :hasGrandparent
)

# hasFather o hasBrother = hasUncle
SubObjectPropertyOf(
  ObjectPropertyChain( :hasFather :hasBrother )
  :hasUncle
)
```

Claves (Keys)

```
# El número de la Seguridad Social identifica
# unívocamente a cada Person
HasKey( :Person () ( :hasSSN ) )

# Turtle:
:Person owl:hasKey ( :hasSSN ) .
```

Cada instancia nombrada de la clase se identifica unívocamente por los valores de esas propiedades

Uso avanzado de datatypes

Restricciones y definiciones de tipos de datos

Definición de datatypes

```
# Edad entre 0 y 150 (facetas de XML Schema)
DatatypeDefinition(
  :personAge
  DatatypeRestriction( xsd:integer
    xsd:minInclusive "0"^^xsd:integer
    xsd:maxInclusive "150"^^xsd:integer
  )
)

# majorAge = personAge AND NOT minorAge
DatatypeDefinition(
  :majorAge
  DataIntersectionOf(
    :personAge
    DataComplementOf( :minorAge )
  )
)
```

Enumeración de datos y restricciones

```
# toddlerAge = {1, 2}
DatatypeDefinition(
  :toddlerAge
  DataOneOf( "1"^^xsd:integer "2"^^xsd:integer )
)

# hasAge es funcional: cada persona tiene una sola edad
FunctionalDataProperty( :hasAge )

# Teenager: edad entre 13 y 19
SubClassOf(
  :Teenager
  DataSomeValuesFrom( :hasAge
    DatatypeRestriction( xsd:integer
      xsd:minExclusive "12"^^xsd:integer
      xsd:maxInclusive "19"^^xsd:integer
    )
  )
)
```

Anotaciones y gestión de ontologías

Metadatos y organización

Anotaciones (Annotations)

```
# Anotar una clase
AnnotationAssertion( rdfs:comment :Person
  "Represents the set of all people." )

# Anotar un axioma
SubClassOf(
  Annotation( rdfs:comment
    "States that every man is a person." )
  :Man :Person
)
```

Las anotaciones **no tienen significado lógico**: son metadatos

Gestión de ontologías

```
# Declarar la ontología con su IRI
Ontology(<http://example.com/owl/families>
  ...
)

# Importar otra ontología
Import(<http://example.org/otherOntologies/
  families.owl>)

# Vincular nombres entre ontologías
SameIndividual( :John otherOnt:JohnBrown )
EquivalentClasses( :Adult otherOnt:Grownup )
EquivalentObjectProperties(
  :hasChild otherOnt:child )
```

OWL 2 DL y OWL 2 Full

Dos formas de interpretar OWL 2

OWL 2 DL

Direct Semantics:

- Subconjunto sintácticamente restringido
- Decidible: los razonadores garantizan respuesta
- Múltiples razonadores de calidad disponibles

OWL 2 Full

RDF-Based Semantics:

- Extensión natural de RDFS
- Indecidible: no hay razonadores completos
- Totalmente reflectivo (todo es un triple)

OWL 2 Profiles

OWL 2 Profiles

OWL 2 completo es muy expresivo pero computacionalmente costoso

Los perfiles son **subconjuntos sintácticos** con buenas propiedades computacionales

Ningún perfil es subconjunto de otro

Todo razonador OWL 2 DL es válido para los tres perfiles (pero puede ser más lento)

OWL 2 EL (Existential Language)

Optimizado para: ontologías grandes con descripciones estructurales complejas: SNOMED-CT, Gene Ontology

Permite: expresiones de clase ricas, cadenas de propiedades, intersección

No permite: negación, disyunción, cuantificación universal, inversas

Complejidad: polinomial (PTIME)

OWL 2 QL (Query Language)

Optimizado para: consultas sobre grandes volúmenes de datos

Basado en DL-Lite: traducción de consultas a SQL

Permite: inversas, jerarquías de propiedades, diagramas ER/UML

No permite: cuantificación existencial a expresión de clase, cadenas de propiedades, igualdad

Complejidad: LogSpace en datos (NLOGSPACE)

OWL 2 RL (Rule Language)

Optimizado para: razonamiento escalable mediante reglas

Compatible con: RDF(S), reglas (forward/backward chaining)

Permite: intersección, cadenas de propiedades, cardinalidad máxima

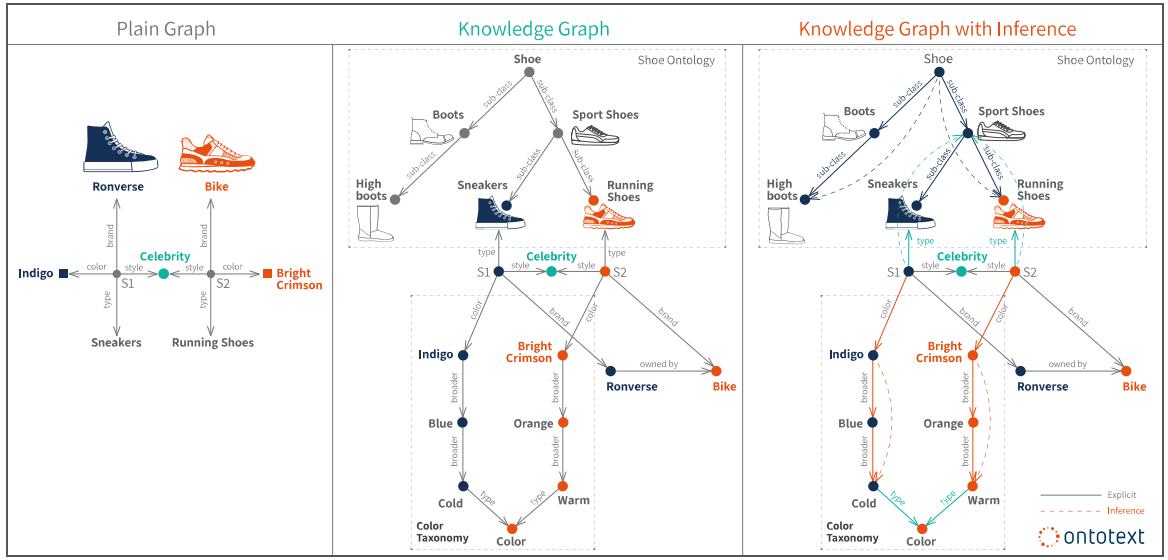
No permite: existenciales que fuercen la existencia de individuos desconocidos

Complejidad: polinomial (PTIME)

Comparación de perfiles OWL 2

Perfil	Caso de uso	Implementación
OWL 2 EL	Clasificación de grandes ontologías	Razonadores especializados (ELK, CEL)
OWL 2 QL	Consultas + ontología ligera	Reescritura de SPARQL → SQL (Ontop)
OWL 2 RL	Enriquecimiento de datos RDF	Motores de reglas (GraphDB)

Knowledge Graphs



OWL y SPARQL

SPARQL consulta el grafo RDF serializado

Por defecto no considera axiomas inferidos

GraphDB: materializa inferencias o razona en tiempo de consulta

SPARQL 1.1: entailment regimes (RDF, RDFS, OWL)

Federated queries: consultar múltiples endpoints con ontologías

Inconsistencias en OWL

Ontología inconsistente: contiene contradicciones lógicas

Clase insatisfacible: no puede tener instancias (equivalente a owl:Nothing)

Causas comunes:

- Clases disjuntas con instancias compartidas
- Restricciones de cardinalidad contradictorias
- Propiedades funcionales con múltiples valores

Explicaciones de inferencias

Justification: conjunto mínimo de axiomas que causa una inferencia

Herramientas: Protégé plugin de explicación

Debugging: identificar axiomas problemáticos

Puede haber múltiples justificaciones para una inferencia

Ontologias publicas

Repositorios de ontologias: LOV, OBO, bioportal, ontobee, etc. Schema

(Estadisticas de uso) ELI